

Отчёт. Векторизация КТ-данных.

Постановка задачи

Дан КТ-скан томографии грецкого ореха (walnut), состоящий из набора срезов в форматах TIFF (скан walnut_agd07 содержит 500 слайсов). Необходимо выполнить сегментацию всего объёма - выделить три функциональных класса объекта: скорлупу, ядро и перегородки.

Результатом сегментации для каждого среза должны являться три бинарные маски для каждого из классов, не пересекающиеся друг с другом.

Данные и предобработка

Исходные данные загружаются в единый трёхмерный массив (VolumeStore). Из всего набора срезов многие содержат пустой фон или краевые артефакты, на которых орех представлен единицами пикселей, не представляющих для нас никакого интереса.

Для ограничения области интереса был реализован графический интерфейс (GUI) `bounds_gui.py`, позволяющий вручную выбрать диапазоны **start** и **end** для всех трёх осей (X, Y, Z). Выбранные границы сохраняются в формат JSON и подаются на вход алгоритму, обрезаая 3D-объём и отсекая заведомо пустые или сильно искаженные краевые слайсы.

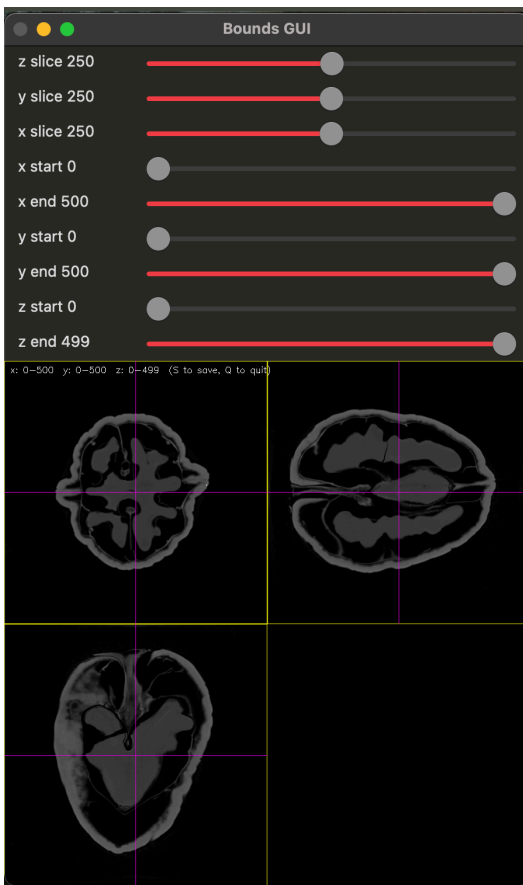


Рис. 1, интерфейс программы выбора границ

Перед передачей в алгоритм сегментации трёхмерный объём проходит глобальную предобработку:

1. Перевод в 8-битный формат (0–255) с min-max нормализацией
2. Усиление контраста для более четкого разделения классов фона и ореха на слайсах
3. В более ранних версиях алгоритма так же в предобработку входило подавление шума на исходном объеме но на сравнении полученных результатов и их метрик разницы не было замечено

Алгоритм сегментации

Сегментация выполняется с помощью класса Segmentator. При сегментации только по одной оси часто встречались проблемы с частично распознанными элементами, из-за чего весь дальнейший алгоритм рушился. Поэтому был разработан алгоритм сегментирования по трем осям, чтобы в полной и максимальной мере вычленять маски классов:

1. Исходный массив сегментируется поочередно с позиции трёх проекций (Z, X, Y), проходя срезы один за другим (_segment_volume_store)
2. При слиянии голосов пиксель классифицируется как структура, если хотя бы в одной из трёх проекций алгоритм проголосовал за её наличие.

Обработка в рамках одного 2D-среза выполняется последовательно и строго иерархично:

1. Скорлупа (Shell)

Для нахождения скорлупы строится гистограмма яркости. Поскольку орех занимает не весь кадр, стандартный метод Оtsu дает смещенный из-за фона результат. Был реализован модифицированный _calc_threshold, вычисляющий порог Оtsu только для пикселей, превышающих определённую яркость (среднее значение кадра, умноженное на bg_fraction). После бинаризации:

- Применяется морфологическое открытие и закрытие, чтобы сгладить края
- Выбирается наибольшая компонента связности с учетом истории размеров предыдущих кадров, чтобы не терять скорлупу при фрагментации
- Дистанционным преобразованием удаляются тонкие линии шума

2. Область интереса и Ядро

- Производится заливка инвертированной маски скорлупы для нахождения "внутренностей" ореха.
- Пиксели, попавшие внутрь, отсекаются по своему, независимому порогу Оtsu. Для предотвращения того, чтобы ядро "прилипло" к границам скорлупы (там часто бывают артефакты отсветов КТ), строится внутренний морфологический барьер: маска скорлупы расширяется вовнутрь на несколько пикселей, блокируя туда рост ядра.
- Ядро также подвергается чистке тонких структур от перегородок.

3. Перегородки и финальная очистка

- Всё, что осталось внутри скорлупы и не классифицировано как ядро, тестируется третьим порогом Оtsu.
- Для избежания геометрических конфликтов (пересечений) вводится буферная зона: уже найденная маска ядра дилатируется. Эта охранная зона принудительно вычитается из перегородок, чтобы ложный "шум" от ядра не стал перегородкой.

Как запускать main.py

Запуск производится через консоль с указанием папки с данными, параметров обрезки и флага оценки, если имеются эталонные маски:

```
uv run main.py
--folder data/walnut_agd07
--bounds walnut_bounds.json
--evaluate
--etalon-dir data/walnut_etalon
```

Оценка результатов и аналитика параметров

1. Визуальные примеры сегментации

Наложение масок на исходный срез позволяет быстро

обнаружить грубые ошибки: скорлупа - красная, ядро - зелёное, перегородки - синие

Оверлей используется для подбора значений параметров запуска на конкретной выборке. С помощью него понятно, какие значения стоит увеличить/уменьшить.

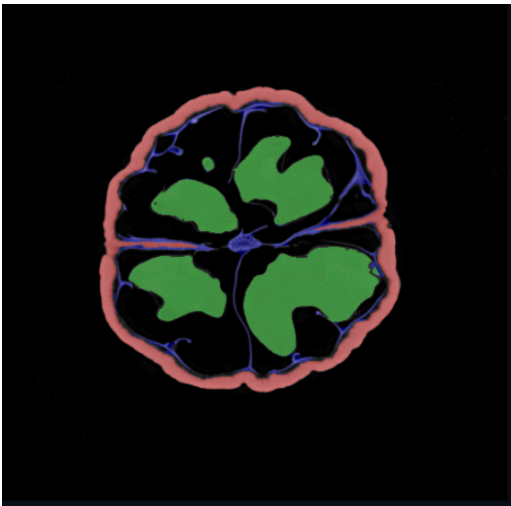


Рис. 2, overlay на слайсе 140

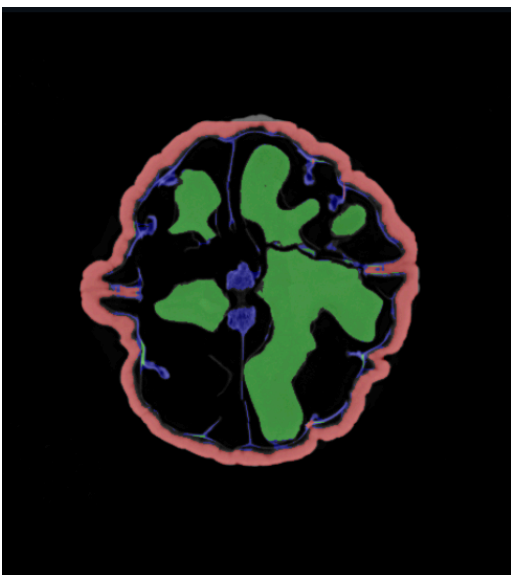


Рис. 3, overlay на слайсе 197

2. Метрики точности сегментации (IoU и Dice)

Для 21 спайса (начиная с 40 по 440 с шагом 20 слайсов) были вручную созданы эталонные маски сегментации для оценки качества работы алгоритма. Реализован SegmentationEvaluator, рассчитывающий метрики сегментации. Результаты для текущей конфигурации алгоритма представлены ниже:

	IoU mean (std)	Dice mean (std)	Precision	Recall
Kernel (Ядро)	0.9136 (0.09)	0.9516 (0.06)	0.9791	0.9280
Shell (Оболочка)	0.8562 (0.03)	0.9222 (0.01)	0.9790	0.8743
Septa (Перегородки)	0.4099 (0.21)	0.5531 (0.19)	0.6822	0.4750

Табл. 1, метрики сегментации

Результаты показывают хорошую степень совпадения по классу ядра и оболочке. Precision для этих классов близок к 0.98: если алгоритм классифицирует пиксель как ядро или скорлупу, его точность очень близка к 100%. Наблюдается незначительное проседание Recall, что означает легкую недосегментацию (границы обрезаются чуть сильнее эталона).

Сложным, как и ожидалось, стал класс перегородок: IoU = 0.41. Из-за своей малой толщины любое отклонение даже в 1-2 пикселя от ручной разметки критически "режет" IoU.

Аналитика входных параметров

Работу Segmentator можно плавно контролировать следующими переменными:

- **bg_fraction** (старт 1.0–1.4) - доля среднего для старта поиска по отсу. При увеличении - порог становится строже, исчезает фоновый шум и меньше деталей попадает в итоговые маски (порог может отсечь из гистограмм полезную информацию), при уменьшении - порог становится более общим, алгоритм Отсу отделяет фон от ореха а не классы между собой.
- **shell_size_fraction** (старт 0.5–0.9) - насколько охотно алгоритм собирает новые крупные компоненты оболочки при объединении нескольких элементов в одну скорлупу, чем меньше - тем меньше компонентов будет собираться, чем больше - появляется риск захватить в маску скорлупы лишние объекты .
- **_thin_max_radius** (старт 5–9) и **_thin_min_area** - параметры очистки мелких и тонких линий. Удаление высоких значений чистит маски, но убирает тонкие частицы перегородок.
- **_kernel_guard_r / _shell_inward_guard_r** (диапазон 2–4) - ключевые параметры для устранения ошибочных определений перегородок у границ. При увеличении маска ядра/ скорлупы не позволяет алгоритму искать перегородку прямо около других уже найденных элементов, что позволяет нам гарантировать, что шум не будет классифицирован как перегородки, но при слишком большом радиусе реальные перегородки исчезнут.

Анализ ошибок и ограничений

1. Разрывы контура скорлупы и проблема замкнутой области

Огромным ограничением текущего пайплайна является алгоритм локализации внутренностей ореха. Область интереса, в которой в дальнейшем ищутся ядро и перегородки, определяется с помощью алгоритма FloodFill (заливка фона с инверсией маски скорлупы). Данный подход строго требует, чтобы контур скорлупы на 2D-срезе был непрерывным и замкнутым. Если скорлупа истончается и контур рвется, заливка вытекает наружу, из-за чего алгоритм считает, что внутренней области на данном срезе просто не существует.

Соответственно, нахождение ядра и перегородок на таком кадре становится технически невозможным.

Хотя сегментация и последующее ансамблирование по трем осям (Z, X, Y) частично исправляют этот недостаток (отсутствующие в разомкнутой на данном слайсе скорлупе ядро и перегородки берутся напрямую из пробегов сегментации по другим осям, где таких проблем уже не было), проблема не решается полностью. На проблемных участках это приводит к возникновению грубых артефактов: например, резкому исчезновению крупных кусков ядра в виде широких горизонтальных или вертикальных срезов.

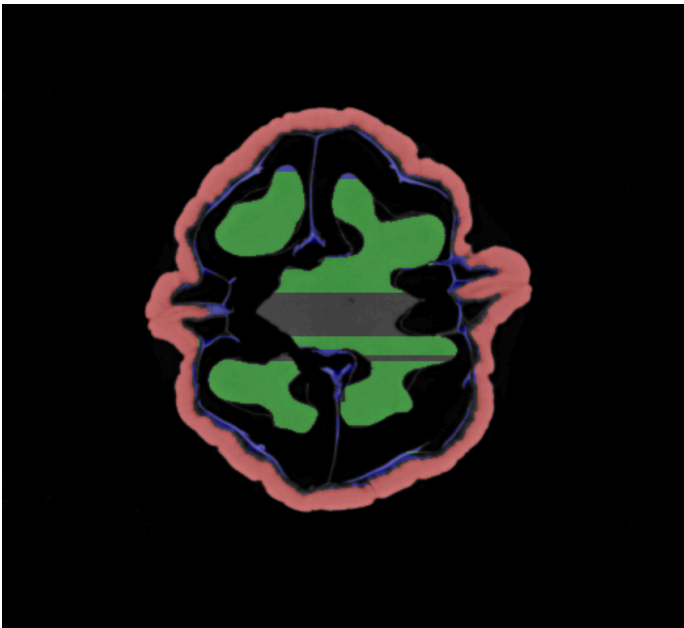


Рис. 4, пример слайса с артефактами (300 слайс)

Возможное решение в будущем

Стоит архитектурно изменить порядок работы пайплайна. Сейчас все три класса (скорлупа, ядро, перегородки) ищутся независимо внутри 2D-среза для всех проекций, а затем сливаются. Более надежным подходом будет:

1. Сначала получить отдельно маску скорлупы по всем трем осям и провести их слияние для получения замкнутого 3D-каркаса.
2. Использовать 3D 'FloodFill' (или покадровый по уже слитому каркасу) для получения цельной и стабильной маски внутренностей.
3. И только после этого, внутри полученной надежной области, искать пороги ядра и перегородок. Это полностью исключит проблему выпадающих кусков внутренностей из-за локальных разрывов оболочки.